

中断延迟时间是指从中断发生到系统获知中断的时间，主要受系统最大关中断时间的影响，关中断的时间越长，中断延迟也就越长。最大关中断时间包含两方面：一是内核最大关中断时间，即内核在执行临界区代码时关闭中断；二是应用关中断时间。关中断最大时间是这两种关中断时间的最大值。

中断响应时间是指从中断发生到开始执行用户中断服务例程的时间；中断恢复时间是指用户中断服务例程结束回到被中断的代码之间的时间。对于可抢占式调度，中断恢复时间还要加上进行任务切换和恢复新的任务上下文的时间；任务响应时间是指从任务对应的中断产生到该任务真正开始运行的时间。

(3) 系统响应时间。系统响应时间指系统在发出处理请求到系统做出应答的时间，即调度延迟，这个时间的大小主要由内核任务调度算法决定。

17.3.2 多任务调度算法

当开发多任务的嵌入式系统应用时，许多普遍性的设计问题随之产生。因为系统的资源是有限的，多个任务执行时，共享和竞争相同的资源不可避免，在可抢占的多任务环境中，资源共享是任务优先级的一个函数，任务的优先级越高，则任务越重要。当访问共享资源时，高优先级的任务先于较低优先级的任务。嵌入式系统中多个任务利用并发执行达到效率最大化，任务之间协同工作也是实际应用的普遍要求。因此，任务之间的通信和同步问题也是设计人员必须考虑的。

1. 调度算法的分类

调度算法根据其时限的性质（软时限还是硬时限）、周期性、可抢占性、静态或动态等准则，可以分为如下几类：

(1) 软时限算法与硬时限算法。实时系统相当复杂，任务优先级的确定与调度并非易事。实时系统大多综合了软实时和硬实时这两种需求，软实时系统只要求任务尽快执行，并不要求在某一特定时间内完成；在硬实时系统中，要求任务不但要正确无误执行，而且还要准时完成。

(2) 周期性与非周期性算法。周期性算法是指调度周期性的任务算法，即每隔 n 个时间单元会执行一次的任务。这 n 个时间单元称为任务周期。非周期性任务是指任务请求处理器的时间是不能预期的。

(3) 可抢占与非抢占算法。可抢占算法是指正在运行的任务可能被其他任务打断，从而放弃 CPU，让其他任务执行；非抢占算法是指任务会一直运行到结束或者等待其他资源而被阻塞，否则不会放弃 CPU。

(4) 静态算法与动态算法。根据任务优先级确定的时机，调度算法分为静态算法和动态算法两类。静态算法是指任务的优先级在设计时就被确定，在任务运行的过程中不会再发生改变。通常，静态调度算法中确定任务优先级的主要依据有执行时间、任务周期和任务的紧迫性。动态算法是指任务的优先级在运行的过程中动态确定，并且会不断地发生变化和更新，该类算法能够完全掌握系统中运行的任务和截止时间、运行时间、优先级，以及到达时间等时间约束，可以灵活地处理变化的系统情况。

(5) 单处理器调度与多处理器调度算法。单处理器算法仅处理一个处理器的情况，多处理器算法可以处理系统中有多个处理器的情况。多处理器算法又分为同质多处理系统和异质多处理系统两种情况。

(6) 在线与离线调度算法。离线调度算法就是运行中使用的调度信息在系统运行之前就被确定，运行的过程中不再变更，离线调度算法具有确定性，但缺少灵活性，适用于那些能够预先知道运行特性，且不易发生变化的应用类型；在线调度算法是指系统运行的调度信息是运行过程中动态收集获取的，如优先级驱动的调度等，该类算法具有最大的灵活性。

当前，大多数内核支持两种普遍的调度算法，即基于优先级的抢占调度和时间轮转调度算法。

- 基于优先级的抢占调度。优先级可以分为静态优先级和动态优先级。应用程序在运行的过程中各任务的优先级固定不变，称为静态优先级。在静态优先级系统中，各任务和它们的时间约束在程序编译时是已知的。应用程序在执行过程中各任务的优先级可以动态改变，称为动态优先级。基于优先级的抢占调度在任何时候运行的任务都是所有就绪任务中具有最高优先级的任务，任务在创建时被赋予了优先级，任务的优先级可以由内核的系统调用动态更改，这使得嵌入式应用对于外部事件的响应更加灵活，从而建立真正的实时响应系统。
- 时间轮转调度算法。为每个任务提供确定份额的CPU执行时间。该调度算法在设计时，应该考虑的因素主要包括CPU的利用率、系统的I/O吞吐量、系统响应时间、公平性和截止时限的满足性等。

2. 单调速率调度算法

对于一个嵌入式实时系统，如果每个任务都能满足时限的要求，则称该系统是可调度的或该系统满足可调度性，也称该系统为健壮的系统（Robust System）。

单调速率调度（Rate Monotonic Scheduling, RMS）算法是一个静态的固定优先级算法，任务的优先级与周期表现为单调函数关系，执行最频繁的任务优先级最高，即任务的周期越短，优先级就越高。RMS是静态调度算法中最有效的算法，如果一组任务能够被任何一种静态调度算法所调度，则在RMS算法下一定是可调度的。RMS算法做了如下假设：

- (1) 所有的任务都是周期性的。
- (2) 任务间不需要同步，没有共享资源，没有任务间的数据交换等问题。
- (3) 系统采用抢占式调度，总是优先级最高且就绪的任务被执行。
- (4) 任务的时限是其下一周期的开始。
- (5) 每个任务具有不随时间变化的定常时间。
- (6) 所有的任务具有同等重要的关键性级别。

要使具有 n 个任务的实时系统中的所有任务都满足硬实时条件，必须使下述的 RMS 定理成立：

$$\sum_i \frac{E_i}{T_i} \leq n \left(2^{1/n} - 1 \right)$$

式中, E_i 是任务 i 的最长执行时间, T_i 是任务 i 的执行周期。也就是说, E_i/T_i 是任务 i 所需的 CPU 时间。基于 RMS 定理, 要所有的任务满足硬实时条件, 所有具有时间要求的任务总的 CPU 利用时间 (或利用率) 应当小于 70%。通常, 作为实时系统设计的一条原则, CPU 利用率应当在 60% ~ 70% 之间。

任务的可调度性通过计算任务的 CPU 利用率, 然后将该利用率与一个可调度的 CPU 利用率上限进行比较而得到。对于实时系统, 普遍使用的可调度性分析方法是速率单调分析法 (Rate Monotonic Analysis, RMA), 该分析法是基于 RMS 定理提出的。例如, 表 17-3 是一个采样系统使用 RMA 进行分析的结果。

表 17-3 任务的相关参数

周期性任务	执行时间	周 期
任务 1	20	100
任务 2	25	150
任务 3	50	300

根据 RMS 定理, 处理器利用率的计算如下:

$$\frac{20}{100} + \frac{25}{150} + \frac{50}{300} \leq 3(2^{1/3} - 1)$$

$$\Rightarrow 53.33\% \leq 77.98\%$$

该问题的利用率是 53.33%, 低于理论边界 77.98%, 定理的条件满足。因此, 系统是可调度的, 即每个任务都满足时限的要求。

3. 时间轮转调度

在时间轮转调度方式中, 当有两个或两个以上就绪任务具有相同的优先级, 且该优先级是就绪任务最高优先级时, 调度程序会依次调度每个任务运行一个小的时间片, 然后再调度另一个任务。每个任务运行完一个时间片, 不管其是否停止或运行尚未结束, 都要释放 CPU 让下一个任务运行, 即相同优先级的任务会得到平等的执行权利, 释放 CPU 的任务被排到同优先级就绪任务队列的尾部, 等待下次调度。

采用时间轮转调度算法时, 时间片的大小选择至关重要, 会影响系统的性能和效率。如果时间片过大, 时间轮转的调度就失去了意义; 如果时间片过小, 任务切换过于频繁, 处理器的开销大, 用于任务运行的有效时间将降低。因此, 对时间片的灵活调整有助于系统性能和效率的提高, 相同优先级的任务可以具有相同的时间片, 不同优先级类别的任务可以具有不同的时间片。

通常, 纯粹的时间轮转调度无法满足实时系统的要求。取而代之的是基于优先级的抢占式时间轮转调度, 对于优先级相同的任务使用时间片获得相等的 CPU 执行时间。如图 17-10 所示, 任务 1、任务 2 和任务 3 具有相同的优先级, 它们按照各自的时间片运行, 任务 2 被更高优先级的任务 4 抢占, 当任务 4 执行完毕后恢复任务 2 的执行。

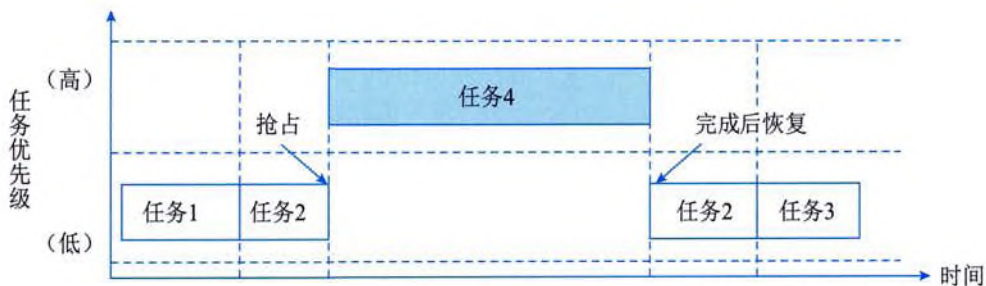


图 17-10 基于优先级的抢占式时间轮转调度

4. 截止时间优先调度

截止时间优先（Earliest Deadline First, EDF）调度算法是指进程的优先级随执行时限变化，即执行时限越靠近，则相对的优先级越高，系统中进程的优先级随时调整。EDF 是一种动态优先级调度算法，根据最大延迟最小化的思想，它基于如下基本定理。

给定一组 n 个独立的任务和一组任意的到达时间，任务可调度性的充分必要条件如下：

$$\sum_i \frac{E_i}{T_i} \leq 1$$

式中， E_i 是任务 i 的最长执行时间， T_i 是任务 i 的执行周期。

EDF 算法要求每个新的就绪任务到达时，都进入就绪任务队列，并根据它们的截止时间排序。如果一个新的已到达任务被插到队列的头部，则当前执行的任务会被抢占。例如，三个任务的抢占调度的情形如图 17-11 所示。

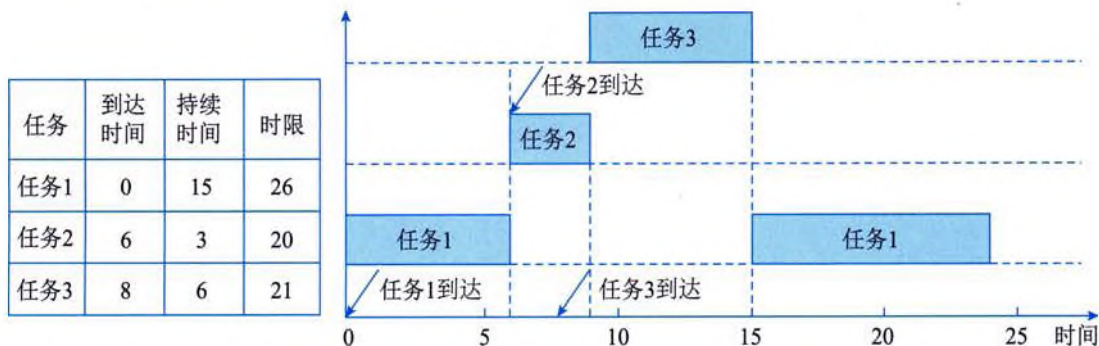


图 17-11 基于截止时间优先调度算法的任务调度图

当任务 1 到达时，它是系统中等待运行的唯一任务，因此立即得到执行。任务 2 在时间 6 到达，由于任务 2 的截止时间（20）早于任务 1 的截止时间（26），因此，任务 2 的优先级比任务 1 的更高，任务 2 抢占任务 1 的 CPU 得到执行。任务 3 在时间 8 到达，由于任务 3 的截止时间晚于任务 2 的截止时间，因此，任务 2 的优先级高于任务 3，任务 2 继续运行，待任务 2 执行结束，任务 3 才能运行。任务 3 执行结束后，任务 1 才能继续运行，在时间 24 执行完毕。

与静态优先级调度算法相比，EDF 算法的优点是其可调度性上限为 100%，即 CPU 的利用率达到了最高。对于任何一组任务，如果 EDF 不能满足其调度性要求，则没有其他算法可以满足这组任务的调度性要求。但在实时系统中，EDF 的实现难度较大，需要在运行中动态确定任务优先级，具有较大的调度开销。

5. 基于分区 (partition) 化的分区 APPS 调度算法

分区 APPS (ARINC plus PPS scheduling) 调度算法是一种满足综合化模块化航空电子系统 (IMA) 的嵌入式操作系统调度算法，是一种静态的任务调度算法，它将处理器时间资源划分成多个独立时间片资源，以供多个分区实体可独立享用属于自己的时间片资源。一个计算资源模块内的所有分区都能按事先规定的时间期限完成工作，则称此计算资源模块所有分区是可调度的。

这里，分区在时间上具有严格的确定性。分区调度主要完成按固定的、基于周期的时间序列进行 CPU 资源的分配，每个分区按照主时间框架分配给它的分区窗口（一个或多个）被调度程序激活。对分区的特定设置而言，调度是固定的。

分区调度原则如下：

(1) 调度单元是分区。

(2) 分区没有优先级。

(3) 分区调度算法预先确定，并按照固定周期重复执行，在每个周期内至少为每个分区分配一个分区窗口。

分区调度一般采用时间优先调度 (TPS)，即周期小分区被优先安排调度。TPS 必须解决分区时间资源的独立性，即确保分区时间隔离的有效性。图 17-12 给出了分区调度的时间调度表。

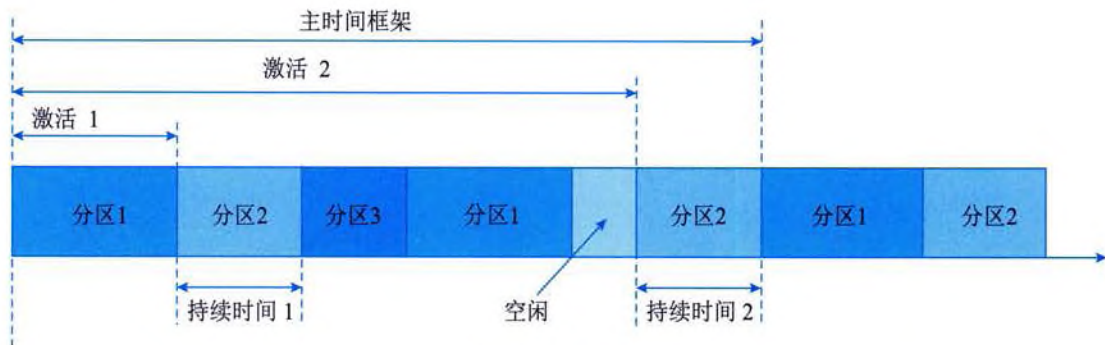


图 17-12 分区调度示意图

分区调度表主要属性包括：Windows ID（时间窗体的标识）、Partition（在时间窗体内被调度的分区标识）、Window offset（针对主时间框架的时间窗体启动时间）、Window Duration（本时间窗体执行的持续时间）。

为了充分利用处理器时间，当 TPS 有闲置的时间片时，系统则会变为优先级调度，这种调度被称为 APPS。图 17-13 给出了 APPS 的调度时序。